Street Report Triage — Full Pitch Reference

Oct 16, 2026 · @Siddhanth Hariharan

1. The problem, and why it's real in Southampton

Southampton residents report broken streetlights, potholes and fly-tipping constantly, often several times for the same problem, because they don't know if it's already been reported. Council officers have to manually read each one, work out what it is, check if it's a duplicate, and send it to the right team — all by hand, on a tight budget, across hundreds of services.

The scale, in real numbers:

- Southampton had **10,948 fly-tipping incidents** in the year to March 2024, the highest of any council in Hampshire
- The rate has been rising every year: **37.6 → 42.7 → 47.1 incidents per 1,000 people** across the last three recorded years
- As of February 2025, there were **755 fly-tipping cases still open** on FixMyStreet for Southampton alone
- 61% of incidents happen on council-owned land, meaning the council is both the reporter's target and the one footing the clean-up bill
- The council separately allocated **£830,000** for dedicated pothole repair crews, showing real budget already being spent on exactly this kind of issue

Why this matters for triage specifically: with volumes this high, every minute an officer spends manually reading, categorising and routing a report is a minute not spent fixing anything. A tool that does the reading and routing automatically, honestly, gives officers back real time on a council budget that's already stretched thin.

2. The project, explained simply

One-liner: we help Southampton City Council street-scene officers triage resident-reported issues by classifying photos, videos and text by type and severity, merging duplicates, and routing each one to the right team.

How it works, including updates back to the resident:

1. A resident contacts a dedicated phone number. An AI-driven conversational agent replies, asking for anything missing — a photo, the exact location, more detail — until it has what it needs. This isn't a general chatbot: it has one job (collect a complete report) and one clear end-state (submit it), which is exactly what keeps it inside the brief's rules rather than falling into the excluded "generic chatbot with no decision" category.
2. Once the agent judges the report complete, it submits the whole conversation (text and any photos) for analysis, rather than analysing message by message.
3. A separate analysis step reads the full conversation and photo, decides the category and severity, confirms the location, and stores the structured report in a database.
4. The resident gets a confirmation notification that their report is logged and being processed, while the classified report is separately routed to the correct council team.
5. As soon as intake is complete (before classification even finishes), the resident gets a "your report has been received" notification, so they know it went through The resident can check progress at any time using their reference number, pulling live status straight from the database Once a council officer marks the report resolved on their dashboard, the resident automatically gets a "your report has been completed" notification, closing the loop

On personal data, honestly: a phone number is personal data, and the brief lists real personal data under things that don't fit the track without an organiser's prior approval. We don't ask for that approval — instead, **the demo uses synthetic, made-up phone numbers**, the same way test reports are synthetic, so no real resident's actual data is ever collected or sent anywhere. The real-world version described in this pitch would need that approval before using genuine phone numbers; the hackathon build itself stays cleanly inside the brief by never touching a real one.

We deliberately do not collect a name. Every function the system needs — receiving a report, classifying it, routing it, sending status updates — only requires the phone number and a reference number, not who the person is. This is a genuine minimal-data-collection choice, not an oversight, and it's a point worth making explicitly when pitching responsibility and trust.

3. Why AI is genuinely necessary, not decoration

The brief is direct about this: "ask yourself what would happen if you removed the AI — if the product would still do its main job, then AI is not at its core."

Our baseline (the dumb version): a simple keyword list — if the text contains "pothole," "road surface," etc., classify it as a pothole. This is genuinely useful to build first, and genuinely limited:

- It cannot read a photo at all, so a report with just an image and no descriptive text gets nothing
- It fails the moment someone describes a problem in their own words instead of the expected keyword ("there's a massive crater, nearly burst my tyre" contains no obvious pothole keyword)
- It can't judge severity — a hairline crack and a tyre-bursting crater both just say "pothole"

What AI adds that the baseline can't: reading a photo directly, understanding descriptions in a resident's own words rather than fixed keywords, and making a judgment call on severity based on what's actually described or shown. We prove the difference by running the same reports through both and showing where they disagree — exactly what the brief asks for.

The honest limit: when confidence is low (an ambiguous photo, vague text with no clear category), the system should say so rather than guess — covered in the failure case in section 6.

4. How to start coding it, in build order

The system breaks into four independent sectors. Each is a small group of classes with one job, same OOP pattern you already know — build and test each sector on its own before wiring them together.

The four sectors:

Sector 1: Conversational Intake Agent — holds the chat with the resident, asks for whatever's missing, and decides when a report is complete.

1. `ConversationSession` — a plain class holding the phone number (or a session ID for the demo), the message history so far, and a status (open or complete)
2. `IntakeAgent` — one method, `handle_message()`: sends the whole conversation so far to Claude, asks it to either reply with a clarifying question or, once there's enough (description, location, a photo if relevant), mark the session complete

`IntakeAgent` always returns structured JSON, not free text — something like `{"status": "ask_more" | "complete" | "incomplete", "reply_text": "...", "summary": {...}}` — so the rest of the pipeline knows reliably what happened, not a guess based on wording A cap on the conversation (e.g. 5 exchanges): if a resident gives vague answers indefinitely, the agent force-completes with whatever's been gathered and marks it `"incomplete"` rather than looping forever — a second honest failure case for the demo, alongside the low-confidence one in section 3

3. For the demo: swap the real phone line for a Streamlit chat box — same conversation logic, no telecoms setup needed this weekend

Sector 2: Classification & Analysis Engine — reads a completed conversation and photo, and turns it into a structured verdict.

1. `KeywordBaselineClassifier` — the dumb version from before, now run against the whole transcript rather than one description
2. `ReportAnalyzer` — sends the full transcript and any photo to Claude, asking for category, severity, and confirmed location back as JSON, same pattern as the single-shot classifier but fed a conversation instead of one message

Sector 3: Database & Storage Layer — stores every completed, classified report so nothing is lost between the chat and an officer's dashboard.

1. `ReportDatabase` — a class wrapping a simple SQLite table or even a JSON file for the weekend: `save_report()`, `get_report()`, `list_reports()`, `update_status()`
2. No separate server needed — this can live entirely inside your Python process for the demo

Sector 4: Notification & Routing Layer — closes the loop back to the resident and forward to the council.

1. `NotificationService` — three trigger points: "received" right after intake completes, "completed" once an officer marks it resolved, and a `get_status(report_id)` method for the resident to check progress anytime. For the demo, each returns or prints a message instead of sending a real SMS.
2. `CouncilRouter` — the same category → team lookup table as before, now reading from the database rather than being called inline

What's genuinely new to learn: holding a multi-turn conversation with an AI (keeping the message history and sending the whole thing back each turn, rather than one message at a time), plus sending an image alongside text (base64-encode it, add it as a second

content block). Everything else — the dataclasses, the lookup tables, the pipeline composition — is OOP you already know.

5. Target user, buyer, and how this scales

Target user: a Southampton City Council street-scene or highways officer, currently reading and routing reports by hand.

Target buyer: Southampton City Council itself — funded either from existing street-scene/highways budget (the £830,000 pothole fund shows real money already allocated to this exact problem area) or justified by officer time saved.

Scaling to a second city: every UK council handles the same basic categories of report — potholes, fly-tipping, streetlights, graffiti — through broadly similar team structures (Highways, Environmental Health, Street Scene). The classifier itself doesn't need to change city to city; only the routing lookup table (which team handles which category) needs updating for a new council's structure. This is explicitly the same path to scale the brief describes for its own "Triage for street reports" example.

Cost per use: one AI call per report (cents, not pounds), plus negligible compute for the distance check and lookup. No ongoing retraining cost, since the AI reasons over the same visual and textual signals regardless of which city it's deployed in.

What doesn't scale automatically: category definitions and severity thresholds may need light tuning per council, since what counts as "urgent" can vary by local policy — worth stating honestly rather than claiming a one-size-fits-all severity scale.

6. Mapped against the Southampton City brief, in order

The one-sentence template required: "We help Southampton City Council street-scene officers triage resident-reported issues across the city using AI that classifies, merges and routes photo, video and text reports."

Who we build for: a council officer (in highways, street scene or environmental health — explicitly listed valid users).

Which theme we fit: streets and public spaces, with overlap into environment and climate (fly-tipping).

What we avoid (the brief's exclusions): not a generic chatbot about Southampton; not a map or prediction with no decision attached; not an existing product with AI bolted on; nothing requiring restricted council systems or real personal data.

Requirement	How we hit it
A real local problem	10,948 fly-tipping incidents, a rising per-capita rate, £830,000 already allocated to potholes — real, current, Southampton-specific
AI at the core	Removing AI leaves a keyword list that can't read photos or handle a resident's own phrasing. Proven against that exact baseline
A product that works on real data	Public FixMyStreet Southampton reports for real examples; any test reports beyond that clearly labelled synthetic
A business case	Buyer: the council, via existing street-scene/highways budget. Second city: same categories, swap the routing table
Honesty about limits	Low-confidence reports flagged for manual review, not guessed

The three questions the brief says our pitch must answer: what local evidence shows the problem is real (section 1 above); who owns and pays (section 5 above); what it takes to run in a second city (section 5 above).

7. Anticipated questions, answered

"Isn't this just FixMyStreet with extra steps?" No — FixMyStreet is where a report goes; our tool is what happens to it after it arrives, automatically reading, classifying and routing it, which today an officer does by hand. We use FixMyStreet's public Southampton reports as real test data, not as our delivery channel.

"Does anyone actually use FixMyStreet for Southampton?" Yes — there were 755 open fly-tipping cases on FixMyStreet for Southampton as of February 2025, and FixMyStreet automatically routes any report to the correct council by location regardless of how well-known the platform is locally.

"How is this different from a simple keyword filter?" A keyword filter can't read a photo and fails on any description that doesn't use the exact expected words. Ours reads both the image and the resident's own phrasing, and we show real cases where the two disagree.

"What happens with duplicate reports?" Reports of the same category within a set distance of each other (currently 50 metres) are flagged as a likely duplicate of the existing one, rather than being treated and routed as a brand new issue.

"What if the AI gets it wrong?" Every classification includes a confidence level; low-confidence cases are flagged for manual review rather than silently routed on a guess.

"Is any personal data involved?" A phone number, yes — needed so we can send status updates back. We deliberately don't collect a name or anything else, since nothing the system does requires one. For the demo, the number is always synthetic/made-up, never a real resident's, keeping us inside the brief's rule against using real personal data without prior approval. Only the conversation and photo, never the number, are sent to the AI.

"What about genuinely dangerous reports, like a severe flood or a hazardous pothole?" The brief requires linking to official guidance rather than writing our own instructions for anything touching safety, health or emergencies. For a "high" severity classification involving immediate danger, the confirmation to the resident includes official guidance — e.g. pointing to the Environment Agency's flood warnings for flooding, or advising they call 999 or the council's emergency highways line for an immediate road hazard — rather than only queuing it for normal council routing.

"Who pays for this?" The council, out of existing street-scene or highways budget — justified by officer time saved rather than a new line item.

"Does this work for every type of report?" Built first for the most common categories — potholes, fly-tipping, streetlights, graffiti — with an "other" category and low-confidence flagging for anything outside that scope, rather than forcing a bad guess.